Step 2e — Il mix finale: proporzioni, bilanciamento, split

Obiettivo

Mettere insieme le coppie delle fonti esistenti (02a), quelle generate con LLM (02b) e il replay (02c) in **un training set e un validation set** con proporzioni decise, etichette bilanciate e nessun documento in comune fra train e val.

Input e output

Input	`data/finetune/raw/<fonte>.jsonl`, `data/finetune/raw/<fonte>+<task>.jsonl` (generate), `data/finetune/replay/<modello>.jsonl`
Output	`data/finetune/mix_<nome>/train.jsonl`, `stats.json`, `decontamination.json` (dev e test non fanno parte del mix: sono gli split congelati dello Step 0)
Codice	`data_prep/make_mix.py` (esiste per le fonti; replay e coppie generate si aggiungono come fonti con lo stesso formato)

1. Cosa fa `make_mix.py`, in ordine

1. Carica le fonti elencate nella ricetta.
2. **Decontamina** contro LLM-AggreFact (claim identici o documenti con ≥ 20% di 13-grammi in comune).
3. Scarta i documenti troppo lunghi (> 24.000 caratteri) e i duplicati.
4. **Validation** (comportamento attuale del codice): il dev originale della fonte, oppure il 5% dei documenti scelti per hash. Con il protocollo dello Step 0 questo passo è sostituito dagli split congelati: il mix legge solo `data/finetune/splits/train/`, e dev e test restano fuori dal mix.
5. Toglie dal train ogni riga che ha un `doc_id` presente nel val (con lo Step 0: in dev o test).
6. Applica, per fonte, il **tetto** di righe e (se richiesto) il **bilanciamento** 50/50 delle etichette.

2. Le ricette

Una ricetta è un JSON `{fonte: {"cap": N, "balance": true|false}}` (cap 0 = tutto). Ricette consigliate (descritte anche in ../dataset_finetuning.md, §6): **senza FactKG in triple** e **senza SciFact** (tenuto come test di dominio nuovo).

`pilot_text` — per le prime corse (~29k coppie)

```
{"minicheck": {"cap": 6000, "balance": true}, "anli": {"cap": 6000, "balance": true},
 "summedits": {"cap": 4000, "balance": true}, "hover": {"cap": 4000, "balance": true},
 "fever": {"cap": 3000, "balance": true}, "vitaminc": {"cap": 3000, "balance": true},
 "averitec": {"cap": 0, "balance": false}}
```

`core_text` — corsa piena (~85k coppie)

```
{"minicheck": {"cap": 0, "balance": false}, "hover": {"cap": 15000, "balance": true},
 "fever": {"cap": 20000, "balance": true}, "anli": {"cap": 15000, "balance": true},
 "vitaminc": {"cap": 15000, "balance": true}, "summedits": {"cap": 0, "balance": false},
 "averitec": {"cap": 0, "balance": false}}
```

`core_text_gen` — con i dati generati (dopo lo step 2b)

`core_text` più, ad esempio:

```
{"minicheck+remove_evidence": {"cap": 4000, "balance": false},
 "hover+remove_evidence": {"cap": 3000, "balance": false},
 "minicheck+contradict_claim": {"cap": 4000, "balance": false},
 "fever+contradict_claim": {"cap": 3000, "balance": false}}
```

Le coppie generate sono tutte negative (label 0): non si bilanciano da sole, ma bilanciano il mix insieme alle loro coppie positive di partenza, che sono già nel mix.

```
python data_prep/make_mix.py --recipe pilot_text.json   # -> data/finetune/mix_pilot_text/
python data_prep/make_mix.py --recipe core_text.json    # -> data/finetune/mix_core_text/
```

3. Come scegliere le proporzioni

Tre principi:

1. **Le fonti più vicine al compito pesano di più.** MiniCheck, ANLI, SummEdits e HoVer (documenti in prosa, claim da verificare per intero, errori sottili) sono ~56% di `core_text`.
2. **Nessuna fonte domina.** Tetti a 15k–20k: anche se VitaminC ha 370k coppie, non deve diventare "il compito". Una fonte troppo grande insegna le sue scorciatoie.
3. **Le fonti piccole ma uniche si prendono tutte** (MiniCheck, SummEdits, AVeriTeC).

Composizione di `core_text` (dal run del 2026-10-08):

Fonte	Train	Quota	Ruolo
FEVER	20.000	23%	Wikipedia, single-hop, NEI
ANLI	15.000	18%	NLI avversariale
VitaminC	15.000	18%	contrasti su numeri e date
MiniCheck	13.627	16%	claim multi-fatto, costruito per il task
HoVer	13.150	15%	multi-hop su più paragrafi
SummEdits	5.809	7%	riassunti, 10 domini
AVeriTeC	2.872	3%	claim reali
Totale	**85.458**		

Il replay non è nella tabella: si aggiunge al momento del rendering come 10–15% degli esempi (circa 10.000–13.000 domande per `core_text`), perché i suoi target dipendono dal modello (02c).

4. Bilanciamento delle etichette

Scelta	Effetto	Quando
50/50 per fonte (default per le fonti grandi)	il modello non impara che "una fonte = quasi sempre no"; la soglia ottimale resta vicina al centro	default
Distribuzione naturale	rispetta le frequenze reali (es. AVeriTeC 70% negativi)	fonti piccole; esperimento D8

La metrica principale è la balanced accuracy: il bilanciamento per fonte è coerente con essa.

5. Il ruolo di `doc_id`

Tre casi in cui due righe condividono il documento:

- più claim sullo stesso documento (MiniCheck, SummEdits: originale e versioni modificate);
- le due revisioni di VitaminC dello stesso caso;
- una coppia originale e quella generata da essa (`remove_evidence`, `contradict_claim` ereditano il `doc_id`).

In tutti i casi le righe devono stare nello **stesso split**. Altrimenti dev o test contengono un documento già visto in training, con un claim diverso, e la loro accuratezza è gonfiata.

6. Controlli sul mix prodotto

`stats.json` riporta righe e quota di positivi per fonte; `decontamination.json` riporta gli scarti per motivo. Da controllare:

Controllo	Valore atteso
Quota di positivi nel train	45–55%
Sovrapposizione `doc_id` fra il train del mix e gli split di dev/test dello Step 0	0
Righe generate (2b) partite da righe non di train	0
Documenti troncati	$< 1\%$ per fonte
Fonti escluse per contaminazione	RAGTruth sempre fuori

7. Lunghezza degli esempi

Fonte	Documento p50 / p95 (caratteri)	$\approx$ token p95
VitaminC	166 / 337	~90
ANLI	313 / 512	~130
AVeriTeC	605 / 2.122	~530
MiniCheck	921 / 1.703	~430
HoVer	1.135 / 2.181	~550
FEVER	1.556 / 4.347	~1.100
SummEdits	2.482 / 11.636	~2.900

Con il prompt (~150–250 token) quasi tutto sta in 4.096 token. I documenti di LLM-AggreFact sono più lunghi (LFQA, MediaSum, MeetingBank): per questo nella corsa piena si sale a 8.192 token e si dà peso a SummEdits e alle coppie generate da documenti lunghi.

Criteri per passare allo step successivo

- Mix `pilot_text` prodotto, con statistiche verificate.
- File di replay e di varianti disponibili per il rendering.